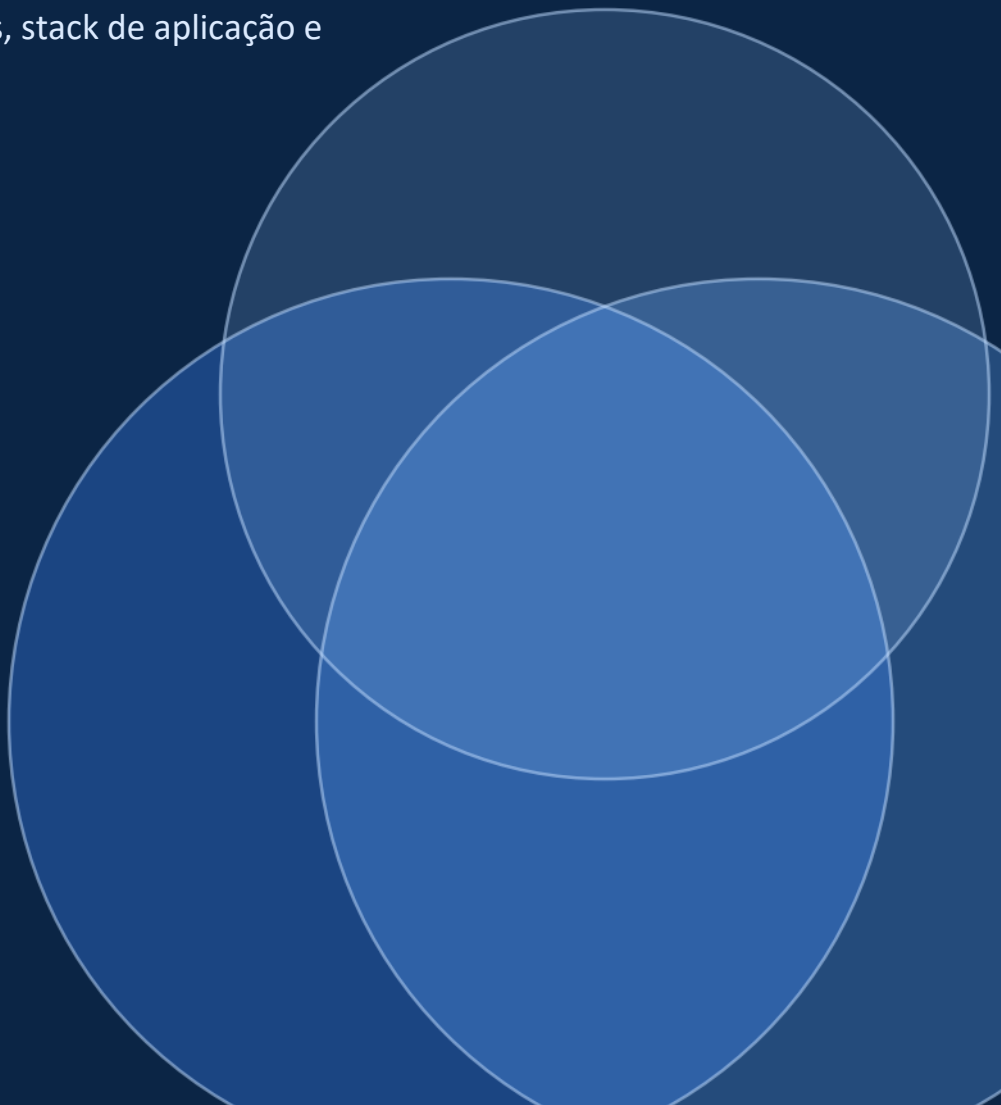
ESTUDOS DE BANCO DE DADOS I

SQL, Workflow e Conceitos Comuns

Consultas com SELECT, JOINs, stack de aplicação e dicionário de dados



Arthur Carvalho Araujo
Material de estudo

Índice

Introdução rápida 3
Conceitos e Funções Fundamentais 3
Principais Categorias de Comandos/Sublinguagens 3
Plataformas RDBMS Populares 3

Abordando consultas 4
DQL 4
Obs: definição dos tipos principais de dados utilizados 4
Exemplo um pouco mais realista 5
Funções e operadores com SELECT 5
JOINS 7

Workflow - Exemplo 9

Dúvidas Conceituais Comuns – Stack, BD, API, Back-End e Front-End 10

Exemplo de código – Back-end e Front-end num único arquivo 12

Dicionário de dados 12

Introdução rápida

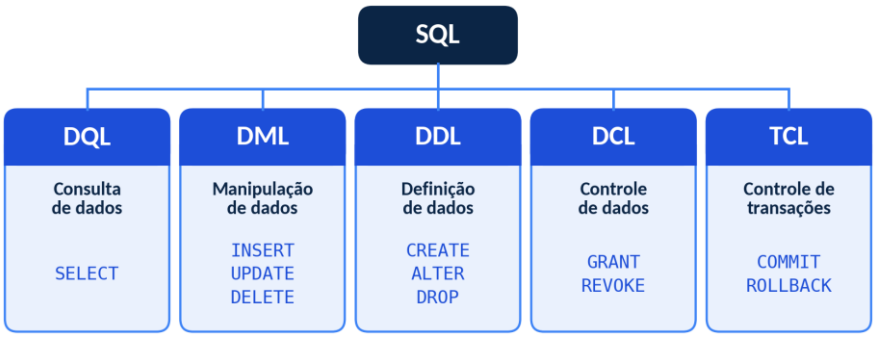
SQL (Structured Query Language, ou Linguagem de Consulta Estruturada) é uma linguagem de programação declarativa e padronizada, utilizada para gerenciar, consultar, atualizar e manipular dados em sistemas de gerenciamento de bancos de dados relacionais (SGBDR).

Conceitos e Funções Fundamentais

- **Linguagem Declarativa**: Você informa ao banco de dados quais dados deseja, em vez de especificar o passo a passo de como encontrá-los.
- **Estrutura de Tabelas**: Os dados são organizados em linhas e colunas dentro de tabelas, que se conectam por meio de chaves primárias e estrangeiras.

Principais Categorias de Comandos/Sublinguagens

- **DQL** (Data Query Language – Linguagem de Consulta de Dados): SELECT, para recuperar dados.
- **DML** (Data Manipulation Language – Linguagem de Manipulação de Dados): INSERT, UPDATE, DELETE, para modificar dados.
- **DDL** (Data Definition Language – Linguagem de Definição de Dados): CREATE, ALTER, DROP, para criar ou alterar estruturas de banco de dados.
- **DCL** (Data Control Language – Linguagem de Controle de Dados): GRANT, REVOKE, para gerenciar permissões de usuário e segurança.
- **DTL / TCL** (Transaction Control Language – Linguagem de Controle de Transações): COMMIT, ROLLBACK, para gerenciar transações.



Sublinguagens do SQL e seus principais comandos

Plataformas RDBMS Populares

RDBMS – inglês: Relational DataBase Management System

SGBDR – português: Sistema Gerenciador de Banco de Dados Relacional

*Geralmente, se fala apenas **SGBD.***

O SQL funciona como um padrão fundamental em diversos sistemas de banco de dados importantes, incluindo:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- Oracle Database
- SQLite

Abordando consultas

Aqui, abordarei sobre **DQL** (Linguagem de Consulta de Dados), pois dentre as outras categorias principais (**DML** - manipulação de dados e **DDL** - definição de dados), esta é a que gera mais dificuldades. Se você aprendeu o suficiente sobre os conceitos de BD (entidades, relacionamentos, chaves primárias, estrangeiras, como referenciar, etc.) **DML** e **DDL** são mais fáceis de inferir, pois são quase que a parte prática dos conceitos, onde você constrói o "esqueleto" do banco (**DDL**) para inserir, apagar ou modificar os dados (**DML**).

> **Dica:** *lembrar o significado das palavras dos comandos ajuda a inferir naturalmente o que ele faz. Ou seja, saber inglês ajuda muito.*

DQL

SELECT: ferramenta para criar query (consulta). Apenas o que é "apresentado na tela" é modificado, sem modificar as tabelas em si.

```
SELECT     tabela.coluna  AS    nome_modificado_para_consulta_apenas
SELECIONE  coluna         COMO  Valdemar
```

AS: essa expressão dá um *alias* (apelido) temporário para a coluna para fim de representação na consulta.

Obs: definição dos tipos principais de dados utilizados

Strings (char, varchar, text, etc.): sequência de caracteres usados para representar texto. Ex.: "carro", "123".

Números (integers, floats, NUMERIC, etc.): usados para operações matemáticas. São armazenados no computador em uma forma binária otimizada para a aritmética.

DATE (Datetime, Timestamp, etc.): usado para armazenar datas (e horas, no caso de Datetime e Timestamp); internamente é guardado como número, mas exibido em formato de data (ex.: "2025-11-05" ou "05/11/2025", conforme o DATESTYLE)

Ex. 2:

```
SET DATESTYLE TO POSTGRES, DMY;
SELECT current_date, current_timestamp;
```

BOOLEAN: tipo de dado que aceita apenas dois valores, TRUE ou FALSE (em alguns SGBDs, representados como 1 ou 0); também pode armazenar NULL (valor desconhecido)

Ex.: Mostre os clientes com cadastro ativo:

```
SELECT nome_cliente FROM cliente WHERE cadastro_ativo = TRUE;
```

Exemplo um pouco mais realista

Mostre, para cada dia da semana, se ele é fim de semana e a receita, da maior para a menor receita:

```
SELECT d.dia_semana, d.fim_de_semana, SUM(f.receita) AS receita
FROM tabela_fato f
JOIN datawarehouse.dimensao_data d ON (f.data_pk = d.data_pk)
GROUP BY d.dia_semana, d.fim_de_semana
ORDER BY receita DESC;
```

> *Não se atenha a detalhes agora, mas a tabela retornaria a coluna* `fim_de_semana` *com* `TRUE` *ou* `FALSE`*.*

Funções e operadores com SELECT

Lembrando, apenas modificam a consulta das tabelas, não as tabelas em si.

> **Dica:** *abuse do CTRL + F. Quando fizer as listas do Guardado, pesquise as sintaxes no dontpad, de preferência salve os dontpads num .txt; isso ajuda demais. No próximo tópico, deixarei um exemplo de workflow para estudos.*

`SELECT * FROM tabela` (selecionar **colunas** de uma tabela)

Se deixado sem filtro (com *****), seleciona todas

`FROM` (de) especificar tabela/coluna/tempo

`WHERE` (onde) filtro para especificar **dados**.

Ex.: `WHERE sexo_pessoa = 'F' OR sexo_pessoa = 'M'`

`WHERE _ LIKE` (onde _ parece) buscar **dados parecidos**.

Ex.: `WHERE LOWER(coluna) LIKE '%azul%'`

`%` se chama *wildcard* (curinga). Aqui, significa “qualquer sequência de caracteres, inclusive nenhum”.

Em ‘%azul%’, significa: “não importa quais caracteres vêm antes ou depois da string azul”

`WHERE _ ILIKE` (WHERE _ INSENSITIVE LIKE) funciona igual o **LIKE**, mas é case insensitive - não diferencia caracteres maiúsculos de minúsculos

`UPPER`, `LOWER` e `INITCAP` MOSTRAM os dados da coluna como MAIÚSCULOS, minúsculos ou Inicial Capitalizada

Ex.: consulte moradores que tem Maria no nome

```
SELECT nome_pessoa, sexo_pessoa, situacao_pessoa
FROM morador
WHERE UPPER(nome_pessoa) LIKE '%MARIA%' ;
```

> *Perceba que, para consultas, usar `ILIKE`, `UPPER`, `LOWER` ou `INITCAP` surte o mesmo efeito, desde que o padrão buscado esteja na mesma caixa da função (`'%MARIA%'` com UPPER, `'%maria%'` com LOWER, `'%Maria%'` com INITCAP); o ILIKE dispensa essa preocupação.*

`OR` (OU) operador lógico OU inclusivo - se a linha tiver qualquer um dos dados no **OR**, ele retorna. Lembre-se da união de conjuntos – se o dado existir em qualquer um dos conjuntos, ele entra no parâmetro

`AND` (E) operador lógico E - só retorna linhas em que ambos os dados forem achados. É a interseção de conjuntos.

`ORDER BY` (ordenar por) ordenar **dados** com base na ordenação de valores em uma coluna

Ex.: `ORDER BY idade;`

`ORDER BY DESC`/`ASC` ordenar de forma descendente/ascendente

`||` `CONCAT()` concatenar/"juntar" 2 ou mais **dados**. Pelo menos um tem que ser string. Todos os dados são convertidos para strings na concatenação.

`::` `CAST()` ("Moldar" **dados**/mudar o tipo de dado)

`TRUNC()` omitir casas decimais de **números**

`ROUND()` omitir casas decimais de **números**, arredondando

`EXTRACT()` extrair pedaços de **data** e **hora**

`INTERVAL` '00' MINUTE/HOUR/MONTH/YEAR seleciona um intervalo de **tempo**, muito usado para somar/subtrair tempo

`current_timestamp` / `now()` "Wed 05 Nov 14:25:04.**663714** 2025 **-03**" timestamp acurado, com fuso horário

`current_date` “Tue 05 Aug 00:00:**00** 2025” timestamp sem fuso horário

JOINS

`INNER JOIN` (junção interna) combinar **linhas** de duas ou mais tabelas que têm valores de chaves em comum.

Ex.1:

```
SELECT
    table1.column1,
    table1.column2,
    table2.column1,
    -- ... other columns
FROM
    table1
INNER JOIN
    table2 ON table1.matching_column = table2.matching_column;
```

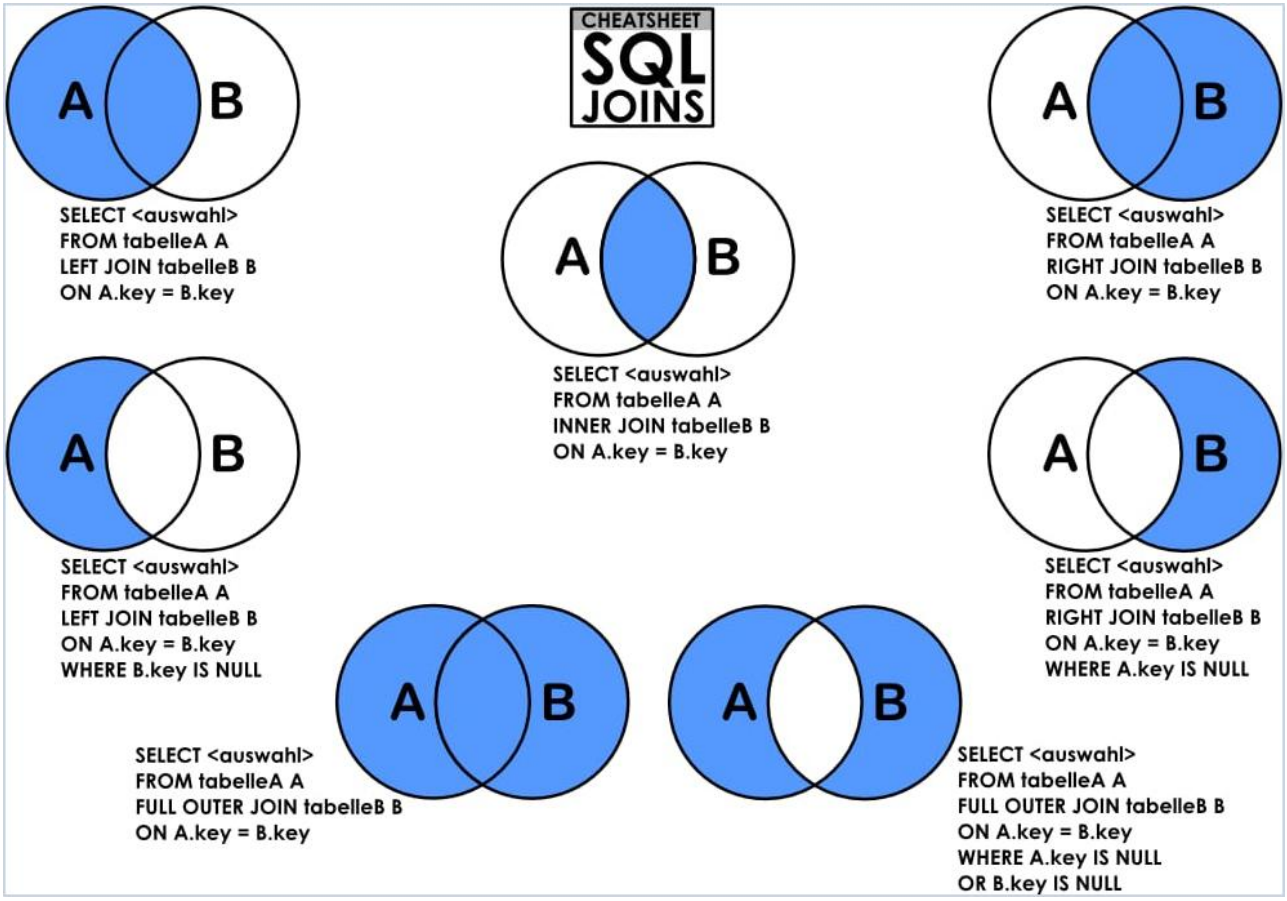
Ex.2:

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    o.order_id,
    o.order_date
FROM
    customers AS c
INNER JOIN
    orders AS o ON c.customer_id = o.customer_id;
```

`USING` (usando) simplifica JOINs quando as colunas possuem exatamente o mesmo nome em ambas as tabelas. Ela substitui a cláusula ON, que é mais prolixa.

Ex.:

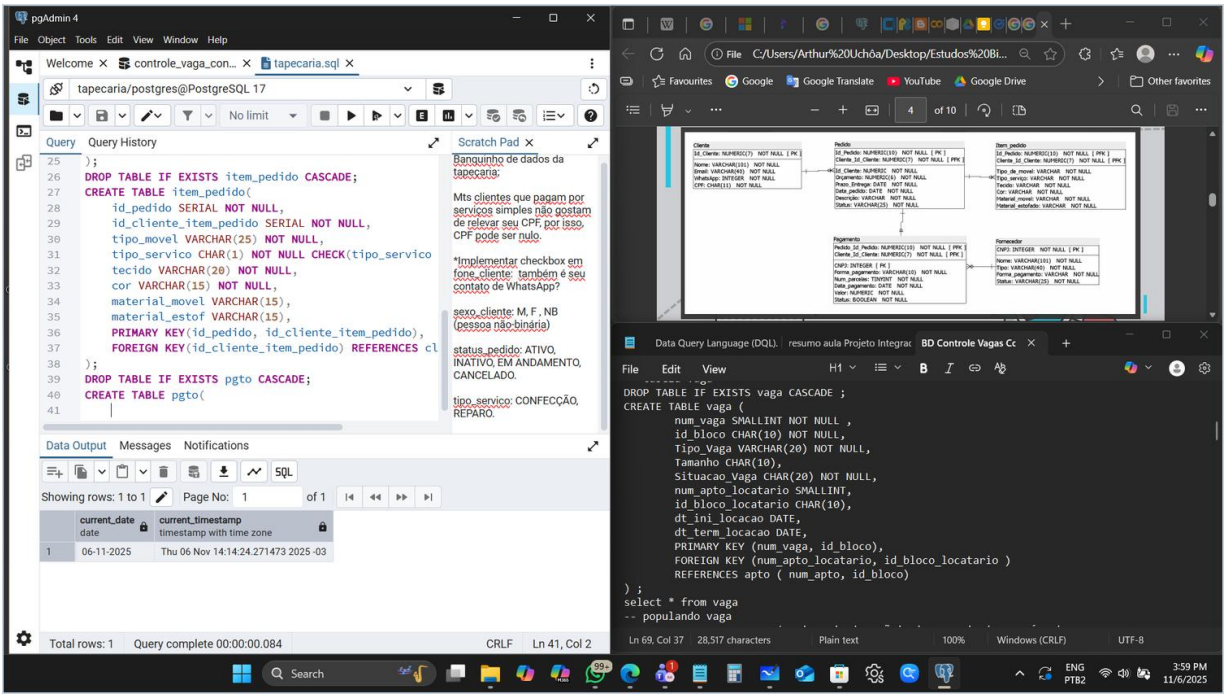
```
SELECT CustomerID, Name, OrderDate
FROM Customers
JOIN Orders
USING (CustomerID);
```



Cheatsheet de SQL JOINs: A e B são as tabelas; a área azul indica as linhas retornadas

No cheatsheet, os termos em alemão significam: `<auswahl>` *= seleção das colunas;* `tabelleA` */* `tabelleB` *= tabela A / tabela B.*

Workflow - Exemplo



Contexto: Instanciando o BD baseado no DER, consultando as sintaxes no dontpad do Guardado

Costumo estudar muito assim, concatenando janelas. Usar atalhos como:

- ctrl + f;
- alt + tab;
- ctrl + tab;
- ctrl + shift + tab;
- Windows + tab;
- ctrl + Home;
- ctrl + End

também ajudam muito, pois a capacidade de aprender rápido faz total diferença neste curso.

Dúvidas Conceituais Comuns – Stack, BD, API, Back-End e Front-End

Ouvi de um aluno que ele ainda não sabia como “tirar” o código em Python do VSCode para mostrar em janelas. Observe estes prints que tirei do meu projeto de PI 1 quando estava no 1º semestre:

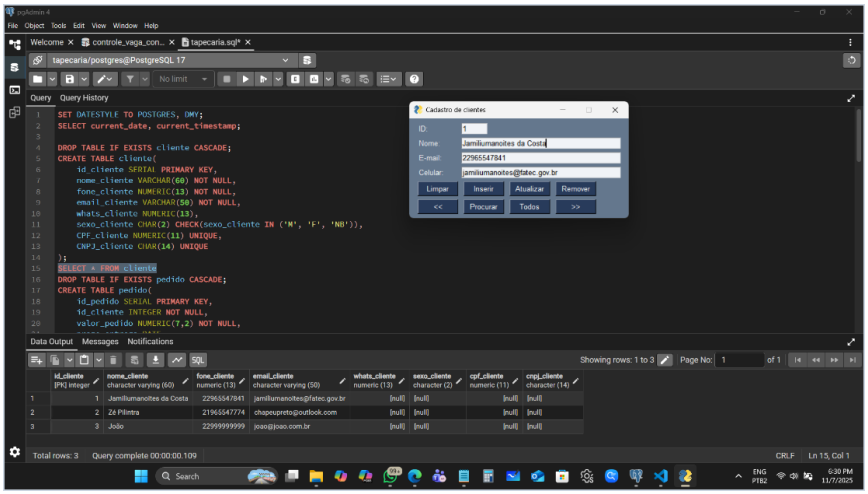


Imagem 1 - BD e Modelo de Front 1

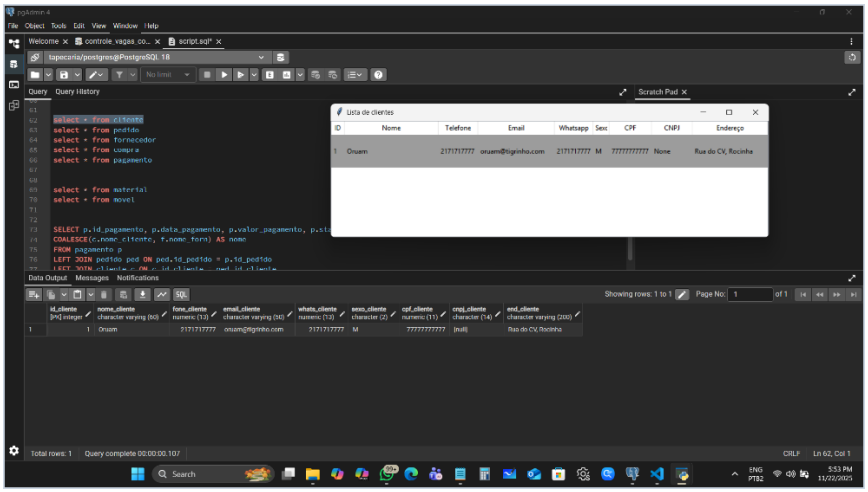


Imagem 2 - BD e Modelo de Front 2

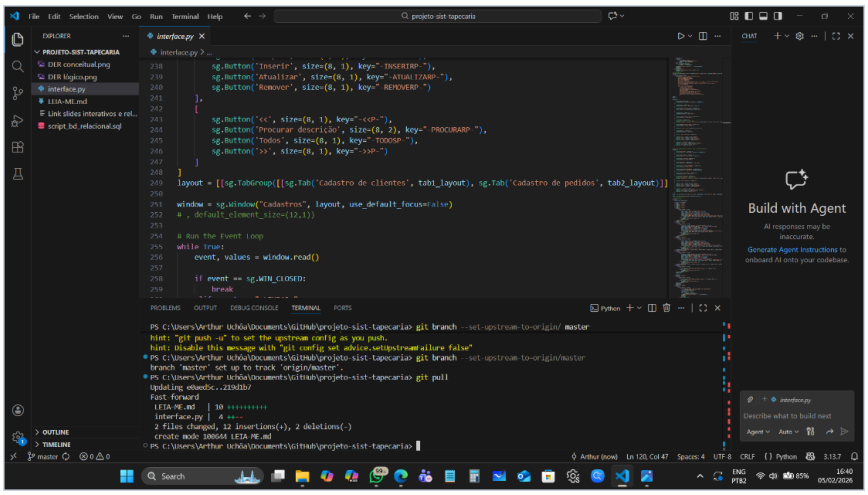


Imagem 3 - Back e Front

Contexto: teste de conexão entre uma instância de um banco de dados no PostgreSQL com o servidor. Aqui, o banco e o servidor (aquele que busca, armazena temporariamente e serve os dados do BD para o cliente) estão configurados na mesma máquina (localhost). A estrutura gráfica da janela 'Cadastro de clientes' (*imagem 1*) e a lógica dos botões (front-end) são geradas pela biblioteca Python PySimpleGUI.

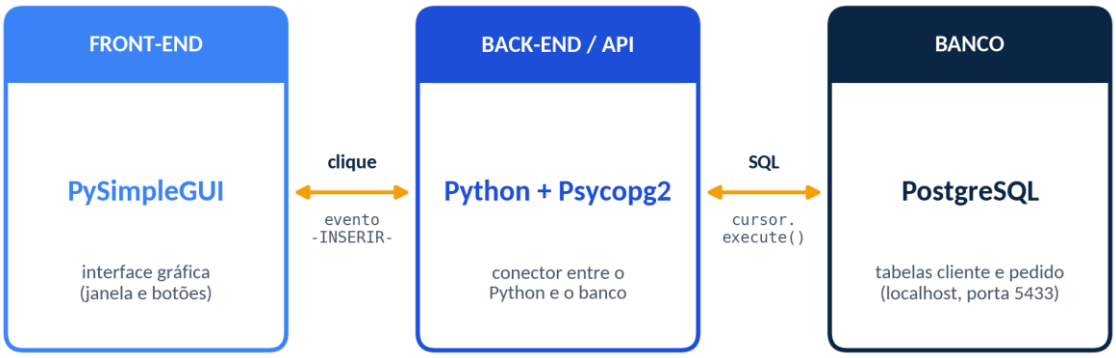
Os botões funcionam pois a ação de clicar em cada botão está associada a uma estrutura lógica (código em Python), e esta estrutura lógica pode ser convertida para conversar com a estrutura do SGBD (código em SQL).

A API (Interface de Programação de Aplicações) é uma ferramenta que faz a conexão entre softwares, intermediando a comunicação. Ela atua como se fosse um garçom. No caso, aqui a API conecta o PySimpleGUI ao banco, converte a ação do clique de cada botão a uma ação CRUD (criar, ler, atualizar, deletar) em SQL no banco. Aqui, a API é implementada pela biblioteca Psycopg2.

Front-end: lib PySimpleGUI (interface gráfica)

Back-end: lib Psycopg2 (conector/API entre o Python e o banco)

Banco: PostgreSQL



Cada camada (ou "escopo") usa um software diferente — isso é uma stack.

Stack do exemplo e o caminho de um clique até o banco

Esta é uma stack extremamente simples. Stack (empilhamento) significa cada software que foi utilizado para cada escopo/nível de desenvolvimento.

Geralmente, cada escopo é armazenado em vários arquivos diferentes em um repositório.

Exemplo de código – Back-end e Front-end num único arquivo

Acesse o arquivo .py disponível no repositório. Ele não irá funcionar pois você teria que criar a instância do BD.

Dicionário de dados

É um metadado que serve para informar as definições dos atributos.

Ex.:

Dicionário de Dados da Entidade Principal "pedido"

Nome do Campo	Tipo de Dado	Tamanho do Campo	Restrição	Descrição
`id_pedido`	Int	10	Primary Key	Id do pedido, autogerado
`id_cliente`	Int	10	Foreign Key, Not Null	Chave estrangeira: referencia a chave primária de cliente
`valor_pedido`	Numeric	7,2	Not Null	Valor, em R$, do pedido
`quant_pedido`	Numeric	7	Not Null	Quantidade total de móveis no pedido, valor padrão: 1
`prazo_entrega`	Date	10		Prazo para finalização do produto em DD/MM/AAAA
`data_pedido`	Date	10	Not Null	Data da criação do pedido em DD/MM/AAAA, autogerada
`descricao_pedido`	Varchar	350	Not Null	Descrição do(s) móvel(eis) a ser(em) confeccionado(s) ou consertado(s)
`status_pedido`	Varchar	50		Status do pedido; valores: Em Andamento, Finalizado, Cancelado; Padrão: Em andamento